



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: Spring, Year: 2026, B.Sc. in CSE (Day)

Mini Bank Management System

Course Title: Object Oriented Programming Lab
Course Code: CSE 202 Section: 251 D1

Students Details:

Name	ID
Jonayet Bogdady	251002011

Submission Date: 23/04/2026
Course Teacher's Name: Kazi Hasnayeem Emad

[For teachers use only: **Don't write anything inside this box**]

Project Proposal Status	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	3
1.1	Overview	3
1.2	Motivation	3
1.3	Problem Definition	3
1.3.1	Problem Statement	3
1.3.2	Complex Engineering Problem	4
1.4	Design Goals/Objectives	4
1.5	Application	5
2	Design, Development, and Implementation of the Project	6
2.1	Introduction	6
2.2	Project Details	6
2.2.1	System Architecture and Data Structure	6
2.2.2	User Interface and Tools	6
2.2.3	Data Persistence and Security	7
2.3	Implementation	7
2.3.1	Library Function	7
2.3.2	User Account Class Code Portion	7
2.3.3	Object Creation Code Portion	8
2.3.4	Auto account number Code portion	8
2.3.5	Button Create Code Portion	8
2.3.6	Save to file Code Portion	8
2.3.7	Load From file Code Potion	9
2.3.8	Login Account Code Potion	9
2.3.9	Login Account Code Potion	11
2.3.10	User Dashboard Code Portion	12
2.3.11	Main Function Code	13

3	Performance Evaluation	14
3.1	Simulation Environment/ Simulation Procedure	14
3.2	Results Analysis/Testing	14
3.2.1	Registration and Login	14
3.2.2	Transaction Accuracy	15
3.3	Project Simulation	15
3.3.1	Create Account Portion	15
3.3.2	Login Account Portion	16
3.3.3	User Dashboard Portion	16
3.3.4	User Transactions History Portion	17
3.4	Results Overall Discussion	17
3.4.1	Complex Engineering Problem Discussion	17
4	Conclusion	18
4.1	Discussion	18
4.2	Limitations	18
4.3	Scope of Future Work	19

Chapter 1

Introduction

1.1 Overview

The Mini Bank Management System is a simple software project developed using Java and Object Oriented Programming concepts. The main purpose of this system is to simulate basic banking operations such as creating an account, depositing money, withdrawing money, checking balance, and viewing transaction history.

This project also includes a graphical user interface (GUI) to make the system user-friendly. The system is designed in a simple way so that users can easily understand and use it. It shows how real banking systems work in a simplified form.

1.2 Motivation

The main reason for choosing this project is to understand how banking systems work in real life. Banking is an important part of our daily life, and almost everyone uses banking services. This project helps to learn important programming concepts such as classes, objects, inheritance, and encapsulation. It also helps to understand how to design a real-life system using programming. Another motivation is to improve problem-solving skills and GUI design skills.

1.3 Problem Definition

1.3.1 Problem Statement

In real life, managing bank accounts manually is difficult and time-consuming. There is a need for a system that can store customer information, manage transactions, and provide quick services.

The problem is to design a system that can:

- Create and manage user accounts
- Handle deposits and withdrawals
- Maintain transaction history
- Provide secure login using PIN

This system should be simple, efficient, and easy to use.

1.3.2 Complex Engineering Problem

Table 1.1: Summary of the attributes touched by the mentioned projects

Name of the P Attributes	Explain how to address
P1: Depth of knowledge required	Requires knowledge of Java, OOP, and GUI design.
P2: Range of conflicting requirements	Needs both security and simplicity for users.
P3: Depth of analysis required	Must analyze user actions like deposit, withdraw, and validation.
P4: Familiarity of issues	Banking system concepts are common but implementation is challenging.
P5: Extent of applicable codes	Uses multiple classes and methods for different operations.
P6: Extent of stakeholder involvement and conflicting requirements	Users want easy interface, system needs secure handling.
P7: Interdependence	Account, transaction, and user data depend on each other.

1.4 Design Goals/Objectives

The main objectives of this project are:

- To create a simple banking system using Java
- To implement Object Oriented Programming concepts
- To design a user-friendly graphical interface
- To provide secure login using PIN
- To manage account balance and transactions correctly
- To store and display transaction history

The system should be reliable, easy to use, and efficient.

1.5 Application

The Mini Bank Management System can be used in many real-life situations. It can be used in small banks, educational purposes, and training systems. This project helps students understand how banking software works.

It can also be used as a base system for developing more advanced banking applications. For example, features like online banking, ATM systems, and mobile banking apps can be added in the future.

In real life, similar systems are used in banks to manage millions of customer accounts. This project represents a simplified version of those systems and helps to understand their basic working process

Chapter 2

Design, Development, and Implementation of the Project

2.1 Introduction

The development of the Mini Bank Management System focused on creating a functional and reliable desktop application that mirrors real-world banking logic. By utilizing the Java programming language, the project incorporates core Object-Oriented Programming (OOP) principles to manage user data and financial transactions. The system is built with a focus on modularity, ensuring that the user interface and the back-end logic work together seamlessly to provide a smooth user experience.

2.2 Project Details

The project is structured around the management of user accounts, where each account is treated as an individual object containing specific attributes.

2.2.1 System Architecture and Data Structure

At the heart of the application is the `UserAccount` class, which stores essential information such as the account holder's name, a unique auto-generated account number, a secure PIN, the account type, and the current balance. To keep track of financial activities, an `ArrayList` is used to store the transaction history for each user. This allows the system to dynamically add records every time a deposit or withdrawal occurs.

2.2.2 User Interface and Tools

The system uses Java Swing and AWT libraries to create a clean Graphical User Interface (GUI). The layout is managed using a `CardLayout`, which allows the program to switch between the Login Page, the Registration Page, and the User Dashboard without opening multiple windows.

2.2.3 Data Persistence and Security

One of the most important parts of this implementation is the file handling system. The program reads from and writes to a text file named bankData.txt. This ensures that when a user creates an account or changes their balance, the information is saved locally and remains available even after the program is closed. Security is maintained by requiring a unique account number and PIN for every login session.

2.3 Implementation

2.3.1 Library Function

```
2  import java.awt.*;
3  import java.io.*;
4  import java.util.ArrayList;
5  import javax.swing.*;
```

Figure 2.1: Library Function Portion

2.3.2 User Account Class Code Portion

```
7  class UserAccount {
8
9      String name, accNum, pin, type, address, phnNum;
10     double balance;
11     ArrayList<String> history;
12
13     public UserAccount(String name, String accNum, String pin,
14         String type, double balance, String address, String phnNum) {
15
16         this.name = name;
17         this.accNum = accNum;
18         this.pin = pin;
19         this.type = type;
20         this.balance = balance;
21         this.address = address;
22         this.phnNum = phnNum;
23
24         history = new ArrayList<>();
25         history.add("Account Created with $" + balance);
26     }
27 }
```

Figure 2.2: User Account Class Code Portion

2.3.3 Object Creation Code Portion

```
29 public class BMS extends JFrame {
30
31     private ArrayList<UserAccount> accounts = new ArrayList<>();
32     private UserAccount currentUser = null;
33
34     private CardLayout cardLayout = new CardLayout();
35     private JPanel mainPanel = new JPanel(cardLayout);
36
37     Font titleFont = new Font(name: "Times New Roman", Font.BOLD, size: 22);
38     Font normalFont = new Font(name: "Times New Roman", Font.PLAIN, size: 14);
39     Color primary = new Color(r: 1, g: 1, b: 1);
40     Color backgroundColor = new Color(r: 52, g: 58, b: 64);
41     Color titleColor = new Color(r: 255, g: 255, b: 255);
```

Figure 2.3: Object Creation Code Portion

2.3.4 Auto account number Code portion

```
60     private String generateAccountNumber() {
61         int num = 10000000 + (int) (Math.random() * 90000000);
62         return String.valueOf(num);
63     }
64
```

Figure 2.4: Auto account number Code portion

2.3.5 Button Create Code Portion

```
65     private JButton styledButton(String text) {
66         JButton btn = new JButton(text);
67         btn.setFocusPainted(b: false);
68         btn.setBackground(primary);
69         btn.setForeground(Color.WHITE);
70         btn.setFont(new Font(name: "Times New Roman", Font.BOLD, size: 18));
71         return btn;
72     }
```

Figure 2.5: Button Create Code Portion

2.3.6 Save to file Code Portion

```
74     private void saveToFile() {
75         try (PrintWriter writer = new PrintWriter(new FileWriter(fileName: "bankData.txt"))) {
76
77             for (UserAccount u : accounts) {
78                 writer.println(
79                     u.name + "," + u.accNum + "," + u.pin + "," + u.type + ","
80                     + u.balance + "," + u.address + "," + u.phnNum + ","
81                     + String.join(delimiter: "|", u.history)
82                 );
83             }
84         } catch (Exception e) {
85             e.printStackTrace();
86         }
87     }
88 }
```

Figure 2.6: Save to file Code Portion

2.3.7 Load From file Code Potion

```
90     private void loadFromFile() {
91         try (BufferedReader reader = new BufferedReader(new FileReader(fileName: "bankData.txt"))) {
92
93             String line;
94
95             while ((line = reader.readLine()) != null) {
96
97                 String[] data = line.split(regex: ",");
98
99                 if (data.length < 7) {
100                     continue;
101                 }
102
103                 UserAccount user = new UserAccount(
104                     data[0], data[1], data[2], data[3],
105                     Double.parseDouble(data[4]), data[5], data[6]
106                 );
107
108                 user.history.clear();
109
110                 if (data.length > 7) {
111                     String[] h = data[7].split(regex: "\\|");
112                     for (String s : h) {
113                         user.history.add(s);
114                     }
115                 }
116                 accounts.add(user);
117             }
118         } catch (FileNotFoundException e) {
119
120         } catch (Exception e) {
121             e.printStackTrace();
122         }
123     }
124 }
```

Figure 2.7: Load From file Code Potion

2.3.8 Login Account Code Potion

```
126     private void initLoginPage() {
127
128         JPanel outer = new JPanel(new GridBagLayout());
129         outer.setBackground(background-color);
130
131         JPanel panel = new JPanel(new GridLayout(rows: 5, cols: 1, hgap: 15, vgap: 15));
132         panel.setPreferredSize(new Dimension(width: 400, height: 250));
133         panel.setBackground(background-color);
134
135         ImageIcon icon = new ImageIcon(filename: "C:\\Users\\User\\Desktop\\New folder (2)\\Java\\Java 2\\jb logo.png");
136         JLabel label1 = new JLabel(icon);
137
138         Image img = icon.getImage();
139         Image scaledImg = img.getScaledInstance(width: 100, height: 90, Image.SCALE_SMOOTH);
140         label1.setIcon(new ImageIcon(scaledImg));
141
142         JLabel title = new JLabel(text: "WELCOME TO JB BANK", SwingConstants.CENTER);
143         title.setForeground(title-color);
144         title.setFont(title-font);
145     }
```

```

146     JTextField accField = new JTextField();
147     accField.setBorder(BorderFactory.createTitledBorder(title: "Account Number"));
148
149     JPasswordField pinField = new JPasswordField();
150     pinField.setBorder(BorderFactory.createTitledBorder(title: "PIN"));
151
152     JPanel btnPanel = new JPanel(new GridLayout(rows: 1, cols: 2, hgap: 10, vgap: 10));
153     btnPanel.setBackground(backgroundColor);
154
155     JButton loginBtn = styledButton(text: "Login");
156     JButton createBtn = styledButton(text: "Create Account");
157
158     btnPanel.add(loginBtn);
159     btnPanel.add(createBtn);
160
161     panel.add(label1);
162     panel.add(title);
163     panel.add(accField);
164     panel.add(pinField);
165     panel.add(btnPanel);
166
167     outer.add(panel);
168
169     loginBtn.addActionListener(e -> {
170
171         String acc = accField.getText();
172         String p = new String(pinField.getPassword());
173
174         for (UserAccount u : accounts) {
175             if (u.accNum.equals(acc) && u.pin.equals(p)) {
176                 currentUser = u;
177                 updateDashboard();
178                 cardLayout.show(mainPanel, name: "dashboard");
179                 return;
180             }
181         }
182
183         JOptionPane.showMessageDialog(this, message: "Wrong Account or PIN");
184     });
185
186     createBtn.addActionListener(e -> cardLayout.show(mainPanel, name: "create"));
187
188     mainPanel.add(outer, constraints: "login");
189 }

```

Figure 2.8: Main Function Code

2.3.9 Login Account Code Potion

```
191 private void initCreateAccountPage() {
192
193     JPanel outer = new JPanel(new GridBagLayout());
194     outer.setBackground(backgroundColor);
195
196     JPanel panel = new JPanel(new GridLayout(rows: 10, cols: 1, hgap: 10, vgap: 10));
197     panel.setPreferredSize(new Dimension(width: 350, height: 500));
198     panel.setBackground(backgroundColor);
199
200     ImageIcon icon = new ImageIcon(filename: "C:\\Users\\User\\Desktop\\New folder (2)\\Java\\Java 2\\jb logo.png");
201     JLabel label1 = new JLabel(icon);
202
203     Image img = icon.getImage();
204     Image scaledImg = img.getScaledInstance(width: 100, height: 90, Image.SCALE_SMOOTH);
205     label1.setIcon(new ImageIcon(scaledImg));
206
207     JLabel ca = new JLabel(text: "Create Account", SwingConstants.CENTER);
208     ca.setFont(new Font(name: "Times New Roman", Font.BOLD, size: 26));
209     ca.setForeground(Color.WHITE);
210
211     JTextField nameF = new JTextField();
212     nameF.setBorder(BorderFactory.createTitledBorder(title: "Full Name"));
213
214     JTextField addressF = new JTextField();
215     addressF.setBorder(BorderFactory.createTitledBorder(title: "Address"));
216
217     JTextField phnF = new JTextField();
218     phnF.setBorder(BorderFactory.createTitledBorder(title: "Phone Number"));
219
220     JTextField pinF = new JTextField();
221     pinF.setBorder(BorderFactory.createTitledBorder(title: "PIN"));
222
223     JComboBox<String> typeF = new JComboBox<>(new String[]{"Savings", "Current"});
224
225     JLabel autoAcc = new JLabel(text: "Account Number: Auto Generated", SwingConstants.CENTER);
226     autoAcc.setFont(new Font(name: "Times New Roman", Font.BOLD, size: 16));
227     autoAcc.setForeground(Color.WHITE);
228
229     JButton saveBtn = styledButton(text: "Register");
230     JButton backBtn = styledButton(text: "Back");
231
232     panel.add(label1);
233     panel.add(ca);
234     panel.add(nameF);
235     panel.add(addressF);
236     panel.add(phnF);
237     panel.add(pinF);
238     panel.add(typeF);
239     panel.add(autoAcc);
240     panel.add(saveBtn);
241     panel.add(backBtn);
242
243     outer.add(panel);
```

```

245     saveBtn.addActionListener(e -> {
246
247         String randomAcc = generateAccountNumber();
248
249         accounts.add(new UserAccount(
250             nameF.getText(),
251             randomAcc,
252             pinF.getText(),
253             (String) typeF.getSelectedItem(),
254             balance: 500.0,
255             addressF.getText(),
256             phnF.getText()
257         ));
258
259         saveToFile();
260
261         JOptionPane.showMessageDialog(this,
262             "Account Created\nAccount Number: " + randomAcc);
263
264         cardLayout.show(mainPanel, name: "login");
265     });
266
267     backBtn.addActionListener(e -> cardLayout.show(mainPanel, name: "login"));
268
269     mainPanel.add(outer, constraints: "create");
270 }

```

Figure 2.9: Create Account Code Portion

2.3.10 User Dashboard Code Portion

```

273 private JLabel ballbl, nameLbl;
274
275 private void initDashboardPage() {
276
277     JPanel outer = new JPanel(new GridBagLayout());
278     outer.setBackground(backgroundColor);
279
280     JPanel panel = new JPanel(new GridLayout(rows: 7, cols: 1, hgap: 10, vgap: 15));
281     panel.setPreferredSize(new Dimension(width: 350, height: 350));
282     panel.setBackground(backgroundColor);
283
284     ImageIcon icon = new ImageIcon(filename: "C:\\Users\\User\\Desktop\\New folder (2)\\Java\\Java 2\\jb logo.png");
285     JLabel label1 = new JLabel(icon);
286
287     Image img = icon.getImage();
288     Image scaledImg = img.getScaledInstance(width: 100, height: 90, Image.SCALE_SMOOTH);
289     label1.setIcon(new ImageIcon(scaledImg));
290
291     nameLbl = new JLabel(text: "", SwingConstants.CENTER);
292     nameLbl.setFont(new Font(name: "Times New Roman", Font.BOLD, size: 20));
293     nameLbl.setForeground(Color.WHITE);
294
295     ballbl = new JLabel(text: "", SwingConstants.CENTER);
296     ballbl.setFont(new Font(name: "Times New Roman", Font.BOLD, size: 16));
297     ballbl.setForeground(Color.WHITE);

```

```

299 JButton depBtn = styledButton(text: "Deposit");
300 JButton witBtn = styledButton(text: "Withdraw");
301 JButton histBtn = styledButton(text: "History");
302 JButton logoutBtn = styledButton(text: "Logout");
303
304 panel.add(label1);
305 panel.add(nameLbl);
306 panel.add(ballLbl);
307 panel.add(depBtn);
308 panel.add(witBtn);
309 panel.add(histBtn);
310 panel.add(logoutBtn);
311
312 outer.add(panel);
313
314 depBtn.addActionListener(e -> {
315     try {
316         double amt = Double.parseDouble(JOptionPane.showInputDialog(message: "Amount:"));
317         currentUser.balance += amt;
318         currentUser.history.add("Deposit $" + amt);
319         saveToFile();
320         updateDashboard();
321     } catch (Exception ex) {
322         JOptionPane.showMessageDialog(this, message: "Invalid Input");
323     }
324 });
325
326 witBtn.addActionListener(e -> {
327     try {
328         double amt = Double.parseDouble(JOptionPane.showInputDialog(message: "Amount:"));
329
330         if (amt <= currentUser.balance) {
331             currentUser.balance -= amt;
332             currentUser.history.add("Withdraw $" + amt);
333             saveToFile();
334             updateDashboard();
335         } else {
336             JOptionPane.showMessageDialog(this, message: "Low Balance");
337         }
338     } catch (Exception ex) {
339         JOptionPane.showMessageDialog(this, message: "Invalid Input");
340     }
341 });
342
343 histBtn.addActionListener(e
344     -> JOptionPane.showMessageDialog(this,
345         String.join(delimiter: "\n", currentUser.history))
346 );
347
348 logoutBtn.addActionListener(e -> cardLayout.show(mainPanel, name: "login"));
349
350 mainPanel.add(outer, constraints: "dashboard");
351 }
352
353 private void updateDashboard() {
354     nameLbl.setText("User: " + currentUser.name + " (" + currentUser.type + ")");
355     ballLbl.setText("Balance: $" + currentUser.balance);
356 }
357

```

Figure 2.10: User Dashboard Code Portion

2.3.11 Main Function Code

```

Run main | Debug main | Run | Debug
358 public static void main(String[] args) {
359     SwingUtilities.invokeLater(BMS::new);
360 }
361 }

```

Figure 2.11: Main Function Code

Chapter 3

Performance Evaluation

3.1 Simulation Environment/ Simulation Procedure

The Mini Bank Management System was tested on a personal computer/Laptop. The system had the following specifications:

- **Processor:** Intel Core i7 (11th Gen)
- **Memory (RAM):** 16 GB
- **Storage:** 512 GB SSD
- **Operating System:** Windows 11 pro (64-bit)
- **Display:** 13.6 inch screen
- **Compiler:** JVM (through Code::VS Code)

The testing procedure was simple. First, the program was compiled using the JVM compiler to check if there were any errors. After that, the program was run and the main menu appeared. Each option from the menu was tested one by one.

3.2 Results Analysis/Testing

Testing was conducted by performing a full cycle of banking operations to ensure the system remains stable under different conditions.

3.2.1 Registration and Login

The first test involved creating multiple accounts. The system successfully generated unique account numbers and saved the details to the local file. The login mechanism correctly identified authorized users and blocked access when an incorrect PIN was entered.

3.2.2 Transaction Accuracy

Deposits and withdrawals were tested with various amounts. The system correctly updated the balance in real-time on the dashboard and appended the correct description to the transaction history. The "Low Balance" alert successfully prevented users from withdrawing more money than they possessed.

3.3 Project Simulation

3.3.1 Create Account Portion

The screenshot shows a web application window titled "JB Bank Management System". The main heading is "Create Account" with a logo above it. The form contains the following fields:

- Full Name: Jonayet Bogdady
- Address: Araihazar
- Phone Number: 0192487****
- PIN: 251002011
- Savings: (dropdown menu)

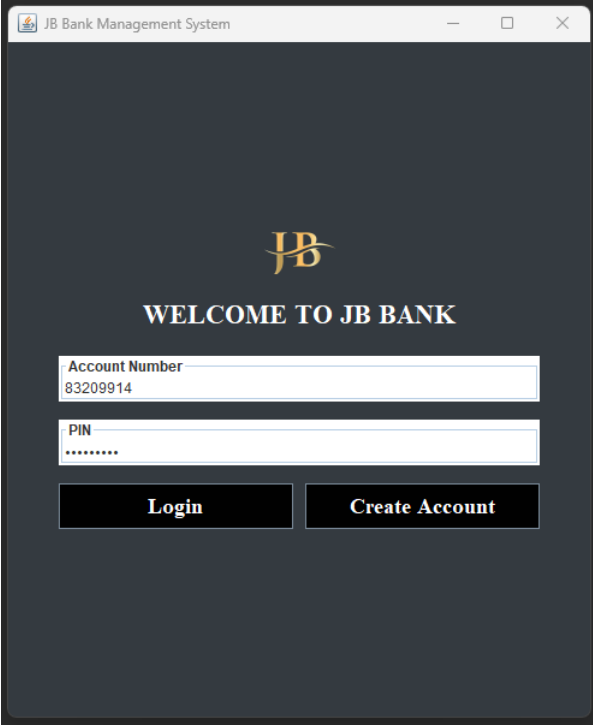
A modal message box is overlaid on the form, displaying:

Message
Account Created
Account Number: 83209914
OK

Below the form, there are two buttons: "Register" and "Back". The text "Account Number: Auto Generated" is visible above the "Register" button.

Figure 3.1: Create Account Portion

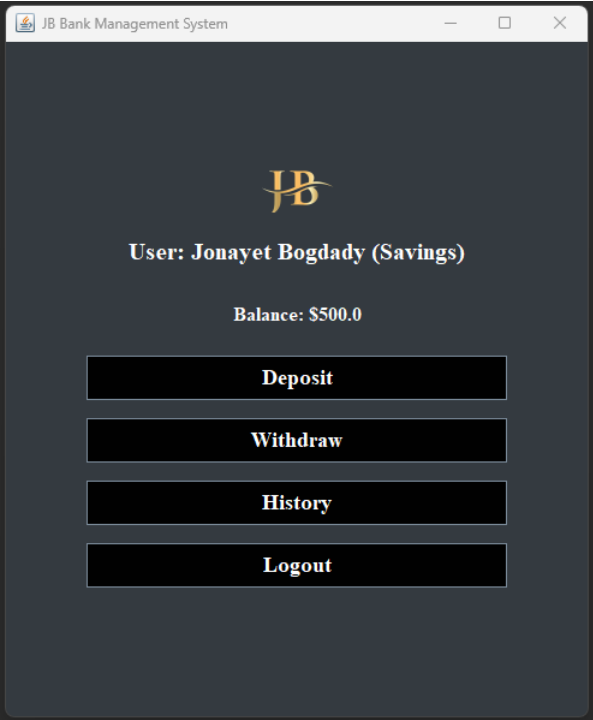
3.3.2 Login Account Portion



The image shows a web browser window titled "JB Bank Management System". The background is dark gray. In the center, there is a gold logo consisting of the letters "JB" with a stylized flourish. Below the logo, the text "WELCOME TO JB BANK" is displayed in white, all-caps font. There are two input fields: the first is labeled "Account Number" and contains the text "83209914"; the second is labeled "PIN" and contains seven asterisks. Below these fields are two buttons: "Login" and "Create Account", both in white text on a dark gray background.

Figure 3.2: Login Account Portion

3.3.3 User Dashboard Portion



The image shows a web browser window titled "JB Bank Management System". The background is dark gray. In the center, there is a gold logo consisting of the letters "JB" with a stylized flourish. Below the logo, the text "User: Jonayet Bogdady (Savings)" is displayed in white, all-caps font. Below this, the text "Balance: \$500.0" is shown in a smaller white font. There are four buttons stacked vertically: "Deposit", "Withdraw", "History", and "Logout", all in white text on a dark gray background.

Figure 3.3: User Dashboard Portion

3.3.4 User Transactions History Portion

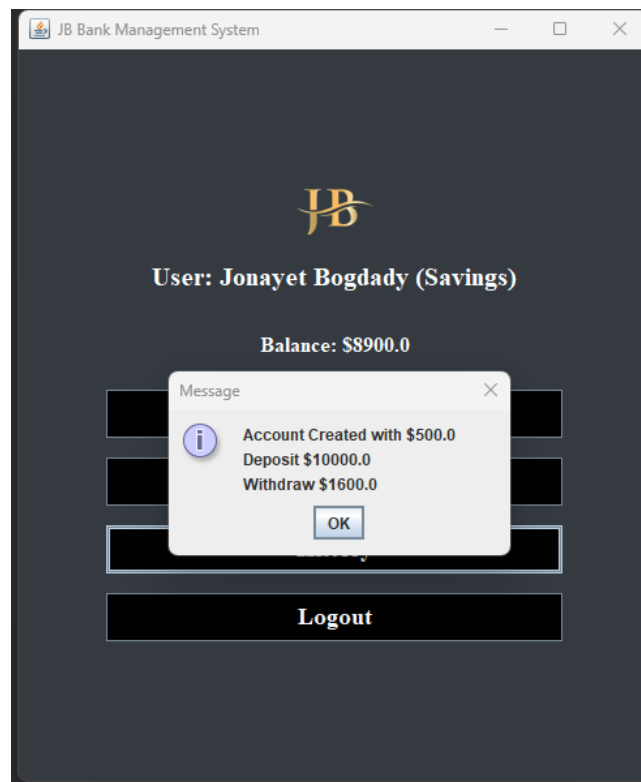


Figure 3.4: User Transactions History Portion

3.4 Results Overall Discussion

The Mini Bank Management System worked as expected in all the tests. The program was able to create an Account and login as user. The deposit function deposit the money and the withdraw function withdraw the money. The history function work as a save the history of an user transaction history.

The program was also able to save the updated user and transaction history of an user list into a text file, which makes the system useful. The system performed well. It did not crash or show errors during testing, even after adding, deleting, and updating many records.

Overall, the project met its main goal of providing a simple and working tool for tish project. The program is small and limited in size but it shows how java programming can be used to solve real-life problems in a simple way.

3.4.1 Complex Engineering Problem Discussion

Even though the project is small, it deals with some important programming challenges. One challenge was storing user Account information in a structured way. This was solved by using a java structure (class User-Account) to keep all related details together, such as name, Address, phone, account type.

Chapter 4

Conclusion

4.1 Discussion

The Mini Bank Management System project successfully fulfills the primary objective of simulating a real-world banking environment through software development. By integrating core Object-Oriented Programming (OOP) logic with a functional Graphical User Interface (GUI), the project demonstrates how technical concepts like encapsulation and inheritance are applied to solve practical data management problems. Throughout the development process, the system proved capable of handling essential banking operations—such as account registration, real-time balance updates, and persistent data storage—with high accuracy. This project serves as a comprehensive bridge between academic theory and the practical requirements of software engineering in the financial sector.

4.2 Limitations

While the project is fully functional as a simulation, it has several limitations that must be acknowledged for a critical analysis:

- **Security Vulnerabilities:** The system currently stores user data in a plain text file (bankData.txt). Since the data is not encrypted, any person with access to the computer can read sensitive information like PINs and account balances.
- **Single-User Environment:** The application is designed as a desktop tool. It lacks networking capabilities, meaning it cannot synchronize data between different multiple users to log in from different locations simultaneously.
- **Basic Authentication:** The security model is limited to a simple account number and PIN check. It does not include modern security features like Multi-Factor Authentication (MFA) or biometric verification.
- **Lack of Concurrency:** The current file-handling logic does not support concurrent writing, which could lead to data corruption if multiple instances of the program tried to update the same file at the same time.

- **Simple Data Structure:** Using a text file for storage becomes inefficient as the number of users grows, leading to slower performance during data retrieval and searching.

4.3 Scope of Future Work

There are several ways to expand and improve this project to make it more robust and professional:

- **Database Integration:** Replacing the text file with a Relational Database Management System (RDBMS) like MySQL or PostgreSQL would significantly improve data security, integrity, and search speed.
- **Advanced Encryption:** Implementing encryption algorithms such as AES (Advanced Encryption Standard) for storing passwords and PINs would ensure that user data remains private even if the storage file is accessed.
- **Network Capabilities:** Transforming the application into a Client-Server model would allow the system to be used across a network, enabling a centralized bank server to handle requests from multiple branch terminals.
- **Enhanced Banking Features:** Future versions could include a fund transfer module for moving money between accounts, a loan management system, and an automated interest calculation feature for savings accounts.
- **Digital Reporting:** Integrating a library like iText or Apache POI would allow the system to generate professional PDF bank statements and transaction receipts for users.
- **Improved UI/UX:** The interface could be upgraded to a more modern look using JavaFX, including charts to visualize spending habits and transaction trends.